

02. Experimental Design

Dynamic Workload Allocation Between Space and Ground

Team [pas encore]

Version 3.0 - September 2026

Table des matières

1	Purpose	2
2	Simulation Environment and Topology	2
3	Hardware Envelope (Developer Environment)	2
4	Workload Profiles (Custom Synthetic Datasets)	3
5	Execution Lifecycle	3
5.1	Queue Telemetry Collection (Added to Phase 3)	4
6	External Benchmark Sources and Validation Protocol	4
6.1	PSPLIB (Project Scheduling Problem Library)	4
6.2	Google / Alibaba Cluster Traces	5
6.3	Skyfield (Orbital Mechanics)	5
6.4	The Merged Scenario (Secondary Validation)	5
7	Methodological Limitations and Controls	5
7.1	Partial Observability (The Forecasting Gap)	5
7.2	The “Cold Start” Reality	5
7.3	Telemetry Uncertainty	6

1 Purpose

The experiment compares five allocation strategies under identical conditions (yelm yo93dou nafshom fi les strategy lkol). This document locks the three experimental conditions that must be held constant so that differences in telemetry are attributable to orchestration alone :

1. the **Simulation Environment** (Topology & Constraints - Section 2),
2. the **Hardware Envelope** (Developer hardware - Section 3),
3. the **Workload Profiles** (Input Datasets - Section 4),

and defines the **execution phases** (Section 5).

2 Simulation Environment and Topology

We define a fixed heterogeneous topology representing a typical orbital computing architecture. This topology is hardcoded and remains identical for all strategies.

TABLE 1 – Fixed Node Configuration with Realistic Parameters

Node ID	Type	CPU	RAM	Energy Cost	Startup Penalty	Sensor Noise
LEO-1	Orbital Satellite	10	32	2.0	5.0 (high)	±5%
LEO-2	Orbital Satellite	10	32	2.0	5.0 (high)	±5%
GS-Tunisia	Ground Station	20	64	0.5	1.0	±1%
Cloud-AWS	Terrestrial Cloud	100	256	0.1	0.0	±0%

Note : “Startup Penalty” represents the energy spike required to power a cold node. “Sensor Noise” simulates the uncertainty of telemetry in orbit.

TABLE 2 – Fixed Link Dynamics

Link	Bandwidth (Mbps)	Availability Schedule
LEO-1 ↔ GS-Tunisia	10	Active for 600s every 5400s (simulates LEO pass)
LEO-2 ↔ GS-Tunisia	10	Active for 600s every 5400s (offset by 2700s)
GS-Tunisia ↔ Cloud-AWS	100	Always Active
LEO-1 ↔ LEO-2 (ISL)	50	Always Active

3 Hardware Envelope (Developer Environment)

All simulation runs are designed to execute on **standard CPU hardware** (laptops or Google Colab CPU instances). No GPU is required for the discrete-event simulation or the initial RL experiments.

Memory and CPU constraints : The simulation is lightweight. The constraint lies in the **simulated environment** (Table 1), not the developer’s hardware. The chosen framework (SimPy) allows millions of simulated events without high memory usage.

4 Workload Profiles (Custom Synthetic Datasets)

To stress different aspects of the strategies, we define four distinct workload profiles. These are not pre-existing datasets ; rather, they are **synthetic task sequences generated by our own task generator** (implemented in `src/env/task_generator.py`). Each profile is carefully designed to isolate a specific performance dimension and to reflect realistic operational scenarios documented in the satellite computing literature

TABLE 3 – Workload Profile Mapping (Final)

Profile Name	Domain	Primary Metric	Task Characteristics
Burst Heavy	CPU-bound	M_T (Makespan)	150 tasks arriving simultaneously. CPU length : 5-15 units. All equal priority.
Energy Critical	Power-constrained	M_E (Efficiency)	100 tasks spaced over 2 hours. High CPU load (15-20 units). Satellite battery limited to 80%.
Radiation-Induced Failures	Hardware/Network reliability	M_R (Resilience)	Failures follow an exponential distribution (SDC : 17 rad, HBM : 44 rad, SEFI : 5000 rad). Tests proactive migration, error detection, and link robustness.
Mixed Priority	Real-time	M_L (Balance)	200 tasks. 20% "Urgent" (require < 100ms), 80% "Batch".

TABLE 4 – Programmatic Handles

Profile	Seed
<code>load_profile("burst")</code>	42
<code>load_profile("energy")</code>	84
<code>load_profile("intermittent")</code>	126
<code>load_profile("mixed")</code>	168

Sample sizes per dataset (number of runs for statistical significance) are fixed during Phase 1 and recorded in `config/experiment.yaml`.

5 Execution Lifecycle

All phases run using the same codebase and fixed configuration.

Phase 1 : Baseline Calibration (Methodological Control)

- **Objective** : Establish baseline telemetry with no dynamic allocation at all.
- **Protocol** : Run Strategies 1 (Always Ground) and 2 (Always Orbital) across all four profiles. This establishes the lower and upper bounds for resource usage.

Phase 2 : Development and Debugging

- **Objective** : Functional implementation of Strategies 3 and 4, followed by training and validation of Strategy 5.
- **Protocol** : Validate state machine transitions (e.g., task states : PENDING → SCHEDULED → EXECUTING → COMPLETED / FAILED). Resolve logic errors in the score calculation and failure detection, then train the RL agent on controlled scenarios. Initial validation uses the "Burst Heavy" profile.

Phase 3 : Production Evaluation and Telemetry Extraction

- **Objective** : Batch execution to extract final performance telemetry.
- **Protocol** : Execute all five strategies across all four workload profiles. Each combination (Strategy x Profile) is executed 10 times with different random seeds for task generation to ensure statistical stability. Metrics are computed from the aggregated logs.

5.1 Queue Telemetry Collection (Added to Phase 3)

To validate our theoretical models against empirical results, the orchestrator records the following time-series data for each node during every simulation run :

- Arrival rate $\lambda(t)$.
- Service rate $\mu(t)$.
- Instantaneous utilization $\rho_i(t) = \lambda_i(t)/\mu_i(t)$.
- Instantaneous queue length $L_q(t)$.

This telemetry allows us to plot the empirical utilization versus waiting-time curve and compare it against the theoretical M/G/1 prediction, providing direct empirical evidence that our dynamic strategies (particularly Strategies 4 and 5) maintain system stability where static strategies fail.

6 External Benchmark Sources and Validation Protocol

While the four synthetic profiles (Section 4) serve as our primary benchmark due to their controllability and interpretability, we complement them with three external sources to ground our results in real-world conditions.

6.1 PSPLIB (Project Scheduling Problem Library)

What it improves : Task structure realism. Dependencies between tasks are not arbitrary.

How we use it : We import standard instances (e.g., `j30.sm` for 30 tasks) and feed their precedence graphs into our orchestrator. A task cannot start until all its predecessors are completed.

Limitation for baselines : Strategies 1 and 2 (Always Ground/Orbital) do not handle dependencies ; they will fail or significantly underperform on this benchmark, highlighting the necessity of dynamic scheduling.

6.2 Google / Alibaba Cluster Traces

What it improves : Arrival pattern realism. Traces capture real-world burstiness, autocorrelation, and micro-peaks that a Poisson process cannot replicate.

How we use it : We sample a subset of timestamps from publicly available traces (e.g., Google Borg) and inject tasks into SimPy at exactly those timestamps.

Reproducibility note : To keep the repository lightweight, we provide a small pre-processed sample (~ 100 arrival timestamps) in `data/google_sample.csv`. Full traces remain optional.

6.3 Skyfield (Orbital Mechanics)

What it improves : Link-loss pattern realism. Instead of manually defining "link drops at $t=500s$ ", we compute visibility windows from real TLE (Two-Line Element) data.

How we use it : The `Link` class's `is_active()` method calls Skyfield to compute the satellite's position relative to the ground station at the current simulation time.

Implementation : Installed via `pip install skyfield`. TLE data is stored in `data/tle.txt`.

6.4 The Merged Scenario (Secondary Validation)

For Phase 3, we combine the three sources into a single `merged_scenario` :

1. Task structures from PSPLIB.
2. Arrival patterns from Google traces.
3. Link availability from Skyfield.

This scenario is run **only once** at the end of Phase 3, as a proof of concept. It does **not** replace the four synthetic profiles, which remain the foundation of our statistical analysis.

7 Methodological Limitations and Controls

To maintain scientific honesty, we explicitly acknowledge the structural limitations of our experimental design :

7.1 Partial Observability (The Forecasting Gap)

Our strategies (including the RL agent) do not have access to future task arrivals or failure times. They are **reactive** or at best **proactive based on history**, but never clairvoyant.

Control : We will report performance separately for scenarios with prior warning versus sudden failures.

7.2 The “Cold Start” Reality

In real satellites, activating a dormant processor or laser link requires a massive current spike. Strategy 2 (Always Orbital) would physically drain the battery rapidly if applied without activation controls in reality.

Control : We include the **Startup Penalty** parameter (Table 1) to penalize strategies that frequently toggle nodes on and off.

7.3 Telemetry Uncertainty

Space sensors are subject to radiation and thermal drift. The RL agent might overfit to perfect data.

Control : We inject Gaussian noise (**Sensor Noise**, Table 1) into the state observations during Phase 3 to test the robustness of Strategies 3, 4, and 5 against imperfect information.